JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class IdentityHashMap<K,V>

java.lang.Object
    java.util.AbstractMap<K,V>
        java.util.IdentityHashMap<K,V>

All Implemented Interfaces:

Serializable, Cloneable, Map<K,V>

```
public class IdentityHashMap<K,V>
extends AbstractMap<K,V>
implements Map<K,V>, Serializable, Cloneable
```

This class implements the Map interface with a hash table, using reference-equality in
place of object-equality when comparing keys (and values). In other words, in an IdentityHashMap, two keys k1 and k2 are considered equal if and only if (k1==k2). (In normal Map implementations (like HashMap) two keys k1 and k2 are considered equal if and only if (k1==null ? k2==null : k1.equals(k2)).)

This class is *not* a general-purpose Map implementation! While this class implements the Map interface, it intentionally violates Map's general contract, which mandates the use of the equals method when comparing objects. This class is designed for use only in the rare cases wherein reference-equality semantics are required.

A typical use of this class is *topology-preserving object graph transformations*, such as serialization or deep-copying. To perform such a transformation, a program must maintain a "node table" that keeps track of all the object references that have already been processed. The node table must not equate distinct objects even if they happen to be equal. Another typical use of this class is to maintain *proxy objects*. For example, a debugging facility might wish to maintain a proxy object for each object in the program being debugged.

This class provides all of the optional map operations, and permits `null` values and the `null` key. This class makes no guarantees as to the order of the map; in particular, it does not guarantee that the order will remain constant over time.

This class provides constant-time performance for the basic operations (`get` and `put`), assuming the system identity hash function (`System.identityHashCode(Object)`) disperses elements properly among the buckets.

This class has one tuning parameter (which affects performance but not semantics): *expected maximum size*. This parameter is the maximum number of key-value mappings that the map is expected to hold. Internally, this parameter is used to determine the number of buckets initially comprising the hash table. The precise relationship between the expected maximum size and the number of buckets is unspecified.

If the size of the map (the number of key-value mappings) sufficiently exceeds the expected maximum size, the number of buckets is increased. Increasing the number of buckets ("rehashing") may be fairly expensive, so it pays to create identity hash maps with a sufficiently large expected maximum size. On the other hand, iteration over collection views requires time proportional to the number of buckets in the hash table, so it pays not to set the expected maximum size too high if you are especially concerned with iteration performance or memory usage.

Note that this implementation is not synchronized. If multiple threads access an identity hash map concurrently, and at least one of the threads modifies the map structurally, it *must* be synchronized externally. (A structural modification is any operation that adds or deletes one or more mappings; merely changing the value associated with a key that an instance already contains is not a structural modification.) This is typically accomplished by synchronizing on some object that naturally encapsulates the map. If no such object exists, the map should be "wrapped" using the `Collections.synchronizedMap` method. This is best done at creation time, to prevent accidental unsynchronized access to the map:

```
Map m = Collections.synchronizedMap(new IdentityHashMap(...));
```

The iterators returned by the `iterator` method of the collections returned by all of this class's "collection view methods" are *fail-fast*: if the map is structurally modified at any time after the iterator is created, in any way except through the iterator's own `remove` method, the iterator will throw a `ConcurrentModificationException`. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.

Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw `ConcurrentModificationException` on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *fail-fast iterators should be used only to detect bugs.*

Implementation note: This is a simple *linear-probe* hash table, as described for example in texts by Sedgewick and Knuth. The array alternates holding keys and values. (This has better locality for large tables than does using separate arrays.) For many JRE implementations and operation mixes, this class will yield better performance than HashMap (which uses *chaining* rather than linear-probing).

This class is a member of the Java Collections Framework.

Since:

1.4

See Also:

System.identityHashCode(Object), Object.hashCode(), Collection, Map, HashMap, TreeMap, Serialized Form

Nested Class Summary

Nested classes/interfaces inherited from class java.util.AbstractMap

AbstractMap.SimpleEntry<K,V>, AbstractMap.SimpleImmutableEntry<K,V>

Nested classes/interfaces inherited from interface java.util.Map

Map.Entry<K,V>

Constructor Summary

Constructors

Constructor	Description
IdentityHashMap()	Constructs a new, empty identity hash map with a default expected maximum size (21).
IdentityHashMap(int expectedMaxSize)	Constructs a new, empty map with the specified expected maximum size.

`IdentityHashMap(Map<? extends K,? extends V> m)`	Constructs a new identity hash map containing the keys-value mappings in the specified map.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
void	`clear()`	Removes all of the mappings from this map.
`Object`	`clone()`	Returns a shallow copy of this identity hash map: the keys and values themselves are not cloned.
boolean	`containsKey(Object key)`	Tests whether the specified object reference is a key in this identity hash map.
boolean	`containsValue(Object value)`	Tests whether the specified object reference is a value in this identity hash map.
`Set<Map.Entry<K,V>>`	`entrySet()`	Returns a `Set` view of the mappings contained in this map.
boolean	`equals(Object o)`	Compares the specified object with this map for equality.
void		

	forEach(**BiConsumer**<? super **K**,? super **V**> action)	Performs the given action for each entry in this map until all entries have been processed or the action throws an exception.
V	**get**(**Object** key)	Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.
int	**hashCode**()	Returns the hash code value for this map.
boolean	**isEmpty**()	Returns `true` if this identity hash map contains no key-value mappings.
Set<**K**>	**keySet**()	Returns an identity-based set view of the keys contained in this map.
V	**put**(**K** key, **V** value)	Associates the specified value with the specified key in this identity hash map.
void	**putAll**(**Map**<? extends **K**,? extends **V**> m)	Copies all of the mappings from the specified map to this map.
V	**remove**(**Object** key)	Removes the mapping for this key from this map if present.
void		

	replaceAll(**BiFunction**<? super **K**,? super **V**,? extends **V**> function)	Replaces each entry's value with the result of invoking the given function on that entry until all entries have been processed or the function throws an exception.
int	**size**()	Returns the number of key-value mappings in this identity hash map.
Collection<**V**>	**values**()	Returns a **Collection** view of the values contained in this map.

Methods inherited from class java.util.AbstractMap

toString

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Map

compute, computeIfAbsent, computeIfPresent, getOrDefault, merge, putIfAbsent, remove, replace, replace

Constructor Detail

IdentityHashMap

```
public IdentityHashMap()
```

Constructs a new, empty identity hash map with a default expected maximum size (21).

IdentityHashMap

```
public IdentityHashMap(int expectedMaxSize)
```

Constructs a new, empty map with the specified expected maximum size. Putting more than the expected number of key-value mappings into the map may cause the internal data structure to grow, which may be somewhat time-consuming.

Parameters:

`expectedMaxSize - the expected maximum size of the map`

Throws:

`IllegalArgumentException - if expectedMaxSize is negative`

IdentityHashMap

```
public IdentityHashMap(Map<? extends K,? extends V> m)
```

Constructs a new identity hash map containing the keys-value mappings in the specified map.

Parameters:

`m - the map whose mappings are to be placed into this map`

Throws:

`NullPointerException - if the specified map is null`

Method Detail

size

```
public int size()
```

Returns the number of key-value mappings in this identity hash map.

Specified by:

`size in interface Map<K,V>`

Overrides:

`size in class AbstractMap<K,V>`

Returns:

`the number of key-value mappings in this map`

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this identity hash map contains no key-value mappings.

Specified by:

`isEmpty` in interface `Map<K,V>`

Overrides:

`isEmpty` in class `AbstractMap<K,V>`

Returns:

`true if this identity hash map contains no key-value mappings`

get

```
public V get(Object key)
```

Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.

More formally, if this map contains a mapping from a key `k` to a value `v` such that `(key == k)`, then this method returns `v`; otherwise it returns `null`. (There can be at most one such mapping.)

A return value of `null` does not *necessarily* indicate that the map contains no mapping
for the key; it's also possible that the map explicitly maps the key to `null`. The `containsKey` operation may be used to distinguish these two cases.

Specified by:

`get` in interface `Map<K,V>`

Overrides:

`get` in class `AbstractMap<K,V>`

Parameters:

`key - the key whose associated value is to be returned`

Returns:

`the value to which the specified key is mapped, or null if this map contains no mapping for the key`

See Also:

`put(Object, Object)`

containsKey

```
public boolean containsKey(Object key)
```

Tests whether the specified object reference is a key in this identity hash map.

Specified by:

`containsKey` in interface `Map<K,V>`

Overrides:

`containsKey` in class `AbstractMap<K,V>`

Parameters:

`key` - possible key

Returns:

`true` if the specified object reference is a key in this map

See Also:

`containsValue(Object)`

containsValue

```
public boolean containsValue(Object value)
```

Tests whether the specified object reference is a value in this identity hash map.

Specified by:

`containsValue` in interface `Map<K,V>`

Overrides:

`containsValue` in class `AbstractMap<K,V>`

Parameters:

`value` - value whose presence in this map is to be tested

Returns:

`true` if this map maps one or more keys to the specified object reference

See Also:

`containsKey(Object)`

put

```
public V put(K key,
             V value)
```

Associates the specified value with the specified key in this identity hash map. If the map previously contained a mapping for the key, the old value is replaced.

Specified by:

`put` in interface `Map<K,V>`

Overrides:

`put` in class `AbstractMap<K,V>`

Parameters:

`key` - the key with which the specified value is to be associated

`value` - the value to be associated with the specified key

Returns:

the previous value associated with key, or null if there was no mapping for key. (A null return can also indicate that the map previously associated null with key.)

See Also:

`Object.equals(Object)`, `get(Object)`, `containsKey(Object)`

putAll

```
public void putAll(Map<? extends K,? extends V> m)
```

Copies all of the mappings from the specified map to this map. These mappings will replace any mappings that this map had for any of the keys currently in the specified map.

Specified by:

`putAll` in interface `Map<K,V>`

Overrides:

`putAll` in class `AbstractMap<K,V>`

Parameters:

`m` - mappings to be stored in this map

Throws:

`NullPointerException` - if the specified map is null

remove

```
public V remove(Object key)
```

Removes the mapping for this key from this map if present.

Specified by:

`remove` in interface `Map<K,V>`

Overrides:

`remove` in class `AbstractMap<K,V>`

Parameters:

`key - key whose mapping is to be removed from the map`

Returns:

`the previous value associated with key, or null if there was no mapping for key. (A null return can also indicate that the map previously associated null with key.)`

clear

```
public void clear()
```

Removes all of the mappings from this map. The map will be empty after this call returns.

Specified by:

`clear` in interface `Map<K,V>`

Overrides:

`clear` in class `AbstractMap<K,V>`

equals

```
public boolean equals(Object o)
```

Compares the specified object with this map for equality. Returns `true` if the given object is also a map and the two maps represent identical object-reference mappings. More formally, this map is equal to another map `m` if and only if `this.entrySet().equals(m.entrySet())`.

Owing to the reference-equality-based semantics of this map it is possible that the symmetry and transitivity requirements of the `Object.equals` contract may be violated if this map is compared to a normal map. However, the `Object.equals` contract is guaranteed to hold among `IdentityHashMap` instances.

Specified by:

`equals` in interface `Map<K,V>`

Overrides:

`equals` in class `AbstractMap<K,V>`

Parameters:

`o - object to be compared for equality with this map`

Returns:

`true if the specified object is equal to this map`

See Also:

`Object.equals(Object)`

hashCode

```
public int hashCode()
```

Returns the hash code value for this map. The hash code of a map is defined to be the sum of the hash codes of each entry in the map's `entrySet()` view. This ensures that `m1.equals(m2)` implies that `m1.hashCode()==m2.hashCode()` for any two `IdentityHashMap` instances `m1` and `m2`, as required by the general contract of `Object.hashCode()`.

Owing to the reference-equality-based semantics of the `Map.Entry` instances in the set returned by this map's `entrySet` method, it is possible that the contractual requirement of `Object.hashCode` mentioned in the previous paragraph will be violated if one of the two objects being compared is an `IdentityHashMap` instance and the other is a normal map.

Specified by:

`hashCode` in interface `Map<K,V>`

Overrides:

`hashCode` in class `AbstractMap<K,V>`

Returns:

the hash code value for this map

See Also:

`Object.equals(Object)`, `equals(Object)`

clone

```
public Object clone()
```

Returns a shallow copy of this identity hash map: the keys and values themselves are not cloned.

Overrides:

`clone` in class `AbstractMap<K,V>`

Returns:

a shallow copy of this map

See Also:

`Cloneable`

keySet

```
public Set<K> keySet()
```

Returns an identity-based set view of the keys contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress, the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` methods. It does not support the `add` or `addAll` methods.

While the object returned by this method implements the `Set` interface, it does *not* obey `Set's` general contract. Like its backing map, the set returned by this method defines element equality as reference-equality rather than object-equality. This affects the behavior of its `contains`, `remove`, `containsAll`, `equals`, and `hashCode` methods.

The `equals` method of the returned set returns `true` only if the specified object is a set containing exactly the same object references as the returned set. The symmetry and transitivity requirements of the `Object.equals` contract may be violated if the set returned by this method is compared to a normal set. However, the `Object.equals` contract is guaranteed to hold among sets returned by this method.

The `hashCode` method of the returned set returns the sum of the *identity hashcodes* of the elements in the set, rather than the sum of their hashcodes. This is mandated by the change in the semantics of the `equals` method, in order to enforce the general contract of the `Object.hashCode` method among sets returned by this method.

Specified by:

`keySet` in interface `Map<K,V>`

Overrides:

`keySet` in class `AbstractMap<K,V>`

Returns:

an identity-based set view of the keys contained in this map

See Also:

`Object.equals(Object)`, `System.identityHashCode(Object)`

values

```
public Collection<V> values()
```

Returns a `Collection` view of the values contained in this map. The collection is backed by the map, so changes to the map are reflected in the collection, and vice-versa. If the map is modified while an iteration over the collection is in progress, the results of the iteration are undefined. The collection supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Collection.remove`, `removeAll`, `retainAll` and `clear` methods. It does not support the `add` or `addAll` methods.

While the object returned by this method implements the `Collection` interface, it does *not* obey `Collection's` general contract. Like its backing map, the collection returned by this method defines element equality as reference-equality rather than object-equality. This affects the behavior of its `contains`, `remove` and `containsAll` methods.

Specified by:

`values` in interface `Map<K,V>`

Overrides:

`values` in class `AbstractMap<K,V>`

Returns:

a collection view of the values contained in this map

entrySet

```
public Set<Map.Entry<K,V>> entrySet()
```

Returns a `Set` view of the mappings contained in this map. Each element in the returned set is a reference-equality-based `Map.Entry`. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress, the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll` and `clear` methods. It does not support the `add` or `addAll` methods.

Like the backing map, the `Map.Entry` objects in the set returned by this method define key and value equality as reference-equality rather than object-equality. This affects the behavior of the `equals` and `hashCode` methods of these `Map.Entry` objects. A reference-equality based `Map.Entry` `e` is equal to an object `o` if and only if `o` is a `Map.Entry` and `e.getKey()==o.getKey() && e.getValue()==o.getValue()`. To accommodate these equals semantics, the `hashCode` method returns `System.identityHashCode(e.getKey()) ^ System.identityHashCode(e.getValue())`.

Owing to the reference-equality-based semantics of the `Map.Entry` instances in the set returned by this method, it is possible that the symmetry and transitivity requirements of the `Object.equals(Object)`

contract may be violated if any of the entries in the set is compared to a normal map entry, or if the set returned by this method is compared to a set of normal map entries (such as would be returned by a call to this method on a normal map). However, the `Object.equals` contract is guaranteed to hold among identity-based map entries, and among sets of such entries.

Specified by:

`entrySet` in interface `Map<K,V>`

Specified by:

`entrySet` in class `AbstractMap<K,V>`

Returns:

a set view of the identity-mappings contained in this map

forEach

```
public void forEach(BiConsumer<? super K,? super V> action)
```

Description copied from interface: `Map`

Performs the given action for each entry in this map until all entries have been processed or the action throws an exception. Unless otherwise specified by the implementing class, actions are performed in the order of entry set iteration (if an iteration order is specified.) Exceptions thrown by the action are relayed to the caller.

Specified by:

`forEach` in interface `Map<K,V>`

Parameters:

action - The action to be performed for each entry

replaceAll

```
public void replaceAll(BiFunction<? super K,? super V,? extends V> function)
```

Description copied from interface: `Map`

Replaces each entry's value with the result of invoking the given function on that entry until all entries have been processed or the function throws an exception. Exceptions thrown by the function are relayed to the caller.

Specified by:

`replaceAll` in interface `Map<K,V>`

Parameters:

function - the function to apply to each entry

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.